

LAB 10

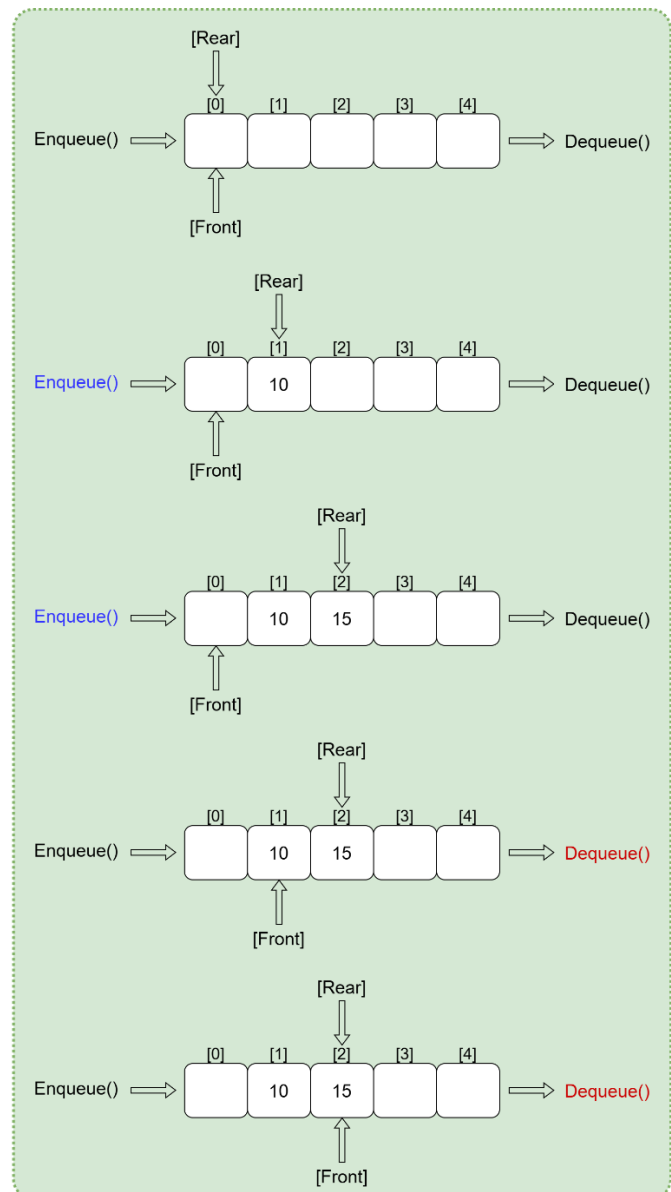
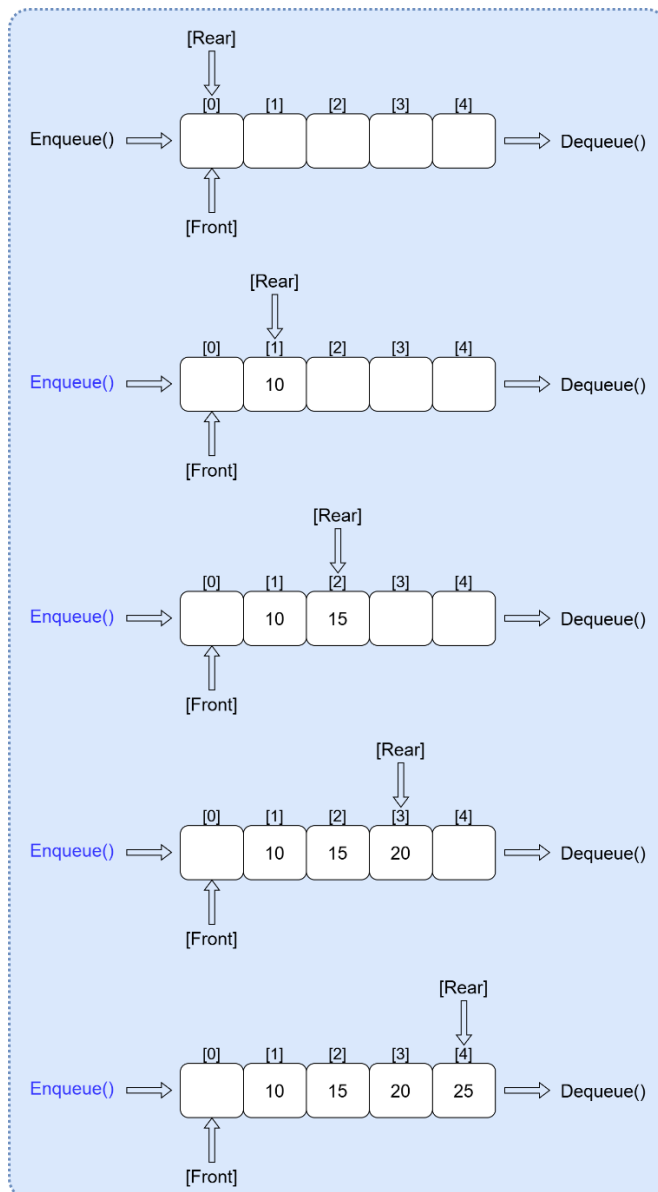
CSE225L



Queue (Array—Based)

In this lab, we will:

- Design and implement the Queue ADT using an array-based circular structure.
- Create the **QueueType** class with methods for Enqueue, Dequeue, and checking the queue's status (IsFull, IsEmpty).
- Test the queue by inserting and removing elements, and handling queue overflow and underflow scenarios.
- Use the queue to generate binary representations of integers from 1 to n .



QUEUE (ARRAY-BASED)

queuetype.h

```
#ifndef QUEUETYPE_H
#define QUEUETYPE_H

const int SIZE = 6;

// Exception class thrown by Enqueue when the queue is full
class FullQueue
{};

// Exception class thrown by Dequeue when the queue is empty
class EmptyQueue
{};

template<class T>
class QueueType
{
private:
    T* data;
    int front;
    int rear;
    int size;
public:
    QueueType();
    QueueType(int s);
    ~QueueType();
    void MakeEmpty();
    bool IsEmpty();
    bool IsFull();
    void Enqueue(T);
    void Dequeue(T &value);
};

#endif // QUEUETYPE_H
```

queuetype.cpp

```
#include "queuetype.h"
#include <iostream>
using namespace std;

template<class T>
QueueType<T>::QueueType()
{
    data = new T[SIZE];
    front = rear = 0;
    size = SIZE;
}

template<class T>
QueueType<T>::QueueType(int s)
{
    size = s + 1;
    data = new T[size];
    front = rear = 0;
}
```

```

template<class T>
QueueType<T>::~~QueueType()
{
    delete [] data;
}

template<class T>
void QueueType<T>::MakeEmpty()
{
    front = rear = 0;
}

template<class T>
bool QueueType<T>::IsEmpty()
{
    return (front == rear);
}

template<class T>
bool QueueType<T>::IsFull()
{
    return ((rear + 1) % size == front); // Full if next position of rear equals front
}

template<class T>
void QueueType<T>::Enqueue(T value)
{
    try
    {
        if (IsFull())
        {
            throw FullQueue();
        }
        else
        {
            rear = (rear + 1) % size; // Move rear to next position
            data[rear] = value;      // Store the new value at the rear
        }
    }
    catch (FullQueue e)
    {
        cout << "Queue Overflow" << endl;
    }
}

template<class T>
void QueueType<T>::Dequeue(T &value)
{
    try
    {
        if (IsEmpty())
            throw EmptyQueue();
        else
        {
            front = (front + 1) % size; // Move front to next position
            value = data[front];        // Retrieve the value from the front
        }
    }
    catch (EmptyQueue e)
    {
        cout << "Queue Underflow" << endl;
    }
}

```

QUEUE (ARRAY-BASED)

TASKS:

Instructions:

- Create the driver file (main.cpp) and perform the following tasks.
- You cannot make any changes to the header (.h) or source (.cpp) files of the **QueueType** class.

Operation	Input Values	Expected Output
Create a queue of integers of size 5		
Print if the queue is empty or not		Queue is Empty
Enqueue four items	5, 7, 4, 2	
Print if the queue is empty or not		Queue is not Empty
Print if the queue is full or not		Queue is not full
Enqueue another item	6	
Print the values in the queue (in the order the values are given as input)		5, 7, 4, 2, 6
Print if the queue is full or not		Queue is Full
Enqueue another item	8	Queue Overflow
Dequeue two items		
Print the values in the queue		4, 2, 6
Dequeue three items		
Print if the queue is empty or not		Queue is Empty
Dequeue an item		Queue Underflow
Take an integer n as input from the user, and use a queue to print the binary values of each integer from 1 to n . Here's how it can be done: ▪ Create an empty queue. ▪ Enqueue the first binary number "1" into the queue. ▪ Run a loop to generate and print n binary numbers: » Dequeue and print the value. » Append "0" to the dequeued value and enqueue it. » Append "1" to the dequeued value and enqueue it.	10	1 10 11 100 101 110 111 1000 1001 1010